Experiment No. 6

Title: Implementation of Rabin-Karp Algorithm

Batch: B3 **Roll No: 16014225801** **Experiment No.:6**

Aim: To study and implement the Rabin–Karp algorithm

Resources needed: Text Editor, C/C++ IDE

Theory:

String pattern matching is a fundamental problem in computer science where the objective is to find occurrences of a pattern string within a larger text string. Naive string matching algorithms compare the pattern with every possible substring of the text, resulting in higher time complexity.

The **Rabin–Karp algorithm** is an optimized string searching algorithm that uses **hashing** to improve performance. Instead of comparing characters directly, it computes hash values of the pattern and substrings of the text. If the hash values match, a character-by-character comparison is performed to confirm the match. This approach significantly reduces unnecessary comparisons.

The algorithm uses a **rolling hash technique**, which allows efficient recalculation of hash values when the search window moves forward in the text. Rabin–Karp is particularly effective when searching for multiple patterns and is widely used in applications such as plagiarism detection and DNA sequence matching.

Advantages of Rabin–Karp Algorithm:

- Efficient average-case performance
- Uses hashing for fast comparison
- Suitable for large text data

Time Complexity:

- Average case: O(n + m)
- Worst case: O(nm)

where n is the length of the text and m is the length of the pattern.

Activity:

Write a program to implement the Rabin–Karp algorithm to search a given pattern in a text string and display the starting index where the pattern is found.

Input format

- First line contains the text string
- Second line contains the pattern string

Output format

- Display the index/indices where the pattern is found in the text

Constraints

Use appropriate constraints

Sample input:

Enter the text: ABCCDDAEFG Enter the pattern: CDD

Sample Output:

Pattern found at index 3

Post Lab:

Hacker Rank Solution:

AI Use Case Implementation:

The code reads daily weather data, converts temperatures into H (hot) or N (normal), and uses the Rabin–Karp algorithm to search for HHH. If found, it identifies a heat wave of three consecutive days with temperatures ≥40°C.

Code:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_ROWS 100000
#define PATTERN "HHH"
#define D 256
#define Q 101

/* Rabin-Karp string matching */
int rabinKarp(char *text, char *pattern)
{
    int n = strlen(text);
    int m = strlen(pattern);

    if (n < m)
        return 0;

    int h = 1;
    int p = 0;
```

```
    int t = 0;

    for (int i = 0; i < m - 1; i++)
        h = (h * D) % Q;

    /* Hash of pattern and first window */
    for (int i = 0; i < m; i++)
    {
        p = (D * p + pattern[i]) % Q;
        t = (D * t + text[i]) % Q;
    }

    for (int i = 0; i <= n - m; i++)
    {
        if (p == t)
        {
            int j;

            for (j = 0; j < m; j++)
            {
                if (text[i + j] != pattern[j])
                    break;
            }

            if (j == m)
                return 1;
        }

        /* Calculate hash for next window */
        if (i < n - m)
        {
            t = (D * (t - text[i] * h) + text[i + m]) % Q;

            if (t < 0)
                t = t + Q;
        }
    }

    return 0;
}


int main()
{
    FILE *file;
    char line[1024];
```

```c
char currentCity[100] = "";
char previousCity[100] = "";

/* Stores H/N sequence for one city */
char *temperaturePattern = (char *)malloc(MAX_ROWS * sizeof(char));

int index = 0;
int heatWaveCount = 0;

file = fopen("india_2000_2024_daily_weather.csv", "r");

if (file == NULL)
{
    printf("Unable to open dataset.\n");
    return 1;
}

/* Skip header */
fgets(line, sizeof(line), file);

while (fgets(line, sizeof(line), file))
{
    char *token;
    char city[100];
    char date[50];
    double maxTemperature;

    /*
        CSV format:

        city,
        date,
        temperature_2m_max,
        temperature_2m_min,
        ...
    */

    token = strtok(line, ",");
    if (token == NULL)
        continue;

    strcpy(city, token);

    token = strtok(NULL, ",");
    if (token == NULL)
```

```
            continue;

        strcpy(date, token);

        token = strtok(NULL, ",");
        if (token == NULL)
            continue;

        maxTemperature = atof(token);

        strcpy(currentCity, city);

        /*
            When city changes, search the previous city's
            temperature sequence using Rabin-Karp.
        */
        if (index > 0 && strcmp(currentCity, previousCity) != 0)
        {
            temperaturePattern[index] = '\0';

            if (rabinKarp(temperaturePattern, PATTERN))
            {
                printf("Heat wave detected in %s\n", previousCity);
                heatWaveCount++;
            }

            index = 0;
        }

        /* Add current day to H/N sequence */
        if (maxTemperature >= 40.0)
            temperaturePattern[index++] = 'H';
        else
            temperaturePattern[index++] = 'N';

        strcpy(previousCity, currentCity);
    }

    /* Check final city */
    if (index > 0)
    {
        temperaturePattern[index] = '\0';

        if (rabinKarp(temperaturePattern, PATTERN))
        {
            printf("Heat wave detected in %s\n", previousCity);
```

```
            heatWaveCount++;
        }
    }

    printf("\nTotal cities with heat waves: %d\n", heatWaveCount);

    free(temperaturePattern);
    fclose(file);

    return 0;
}
```

Output:

```
(base) bhumit@bhumits exp6 % "/Users/bhumit/Desktop/Sem5/cpl/code/exp6/use case"
Heat wave detected in Delhi
Heat wave detected in Kolkata
Heat wave detected in Chennai
Heat wave detected in Hyderabad
Heat wave detected in Ahmedabad
Heat wave detected in Pune
Heat wave detected in Jaipur
Heat wave detected in Lucknow
```

Conclusion:

The Rabin–Karp algorithm was successfully implemented using rolling hash techniques to detect heat waves in daily weather data. The program identified cities having three consecutive days with temperatures of at least 40°C, demonstrating efficient pattern matching on real-world datasets.

References:

1. https://www.hackerearth.com/practice/algorithms/string-algorithm/string-searching/practice-problems/algorithm/random-keyboard-7fac53bc/
2. T.H. Coreman ,C.E. Leiserson,R.L. Rivest, and C. Stein, " Introduction to algorithms", 3rd Edition 2009, Prentice Hall India Publication
3. Antti Laaksonen, “Guide to Competitive Programming”, Springer 2018

Code Drive link
https://drive.google.com/drive/folders/1c9mCFdPtIEW3TAs0tWpC9x2xJQwQ1E3j?usp=sharing